



PRACTICAL LOGBOOK

BIT3208: Advanced Web Design and Development

ShopMart – PHP/MySQL E-Commerce System

Student Name	Stephen Otiende
Admission Number	BBIT/2018/36596
Course / Class	Bachelor of Business Information Technology
Unit Code	BIT3208
Unit Name	Advanced Web Design and Development
Lecturer Name	Mr. Nyoro
Semester / Year	Semester 2 / 2025–2026
GitHub / Portfolio	https://github.com/Otyzo/Advanced-web-design-Tasks

Table of Contents

Week 1: Local Environment Setup	3
Week 2: Wireframes and GUI Design	7
Week 3: JavaScript and PHP Basics	12
Week 4: User Authentication	16
Week 5: Product and Cart System	20

Week 1: Local Environment Setup

Title

Installation and Testing of Local Development Environment

Required Evidence

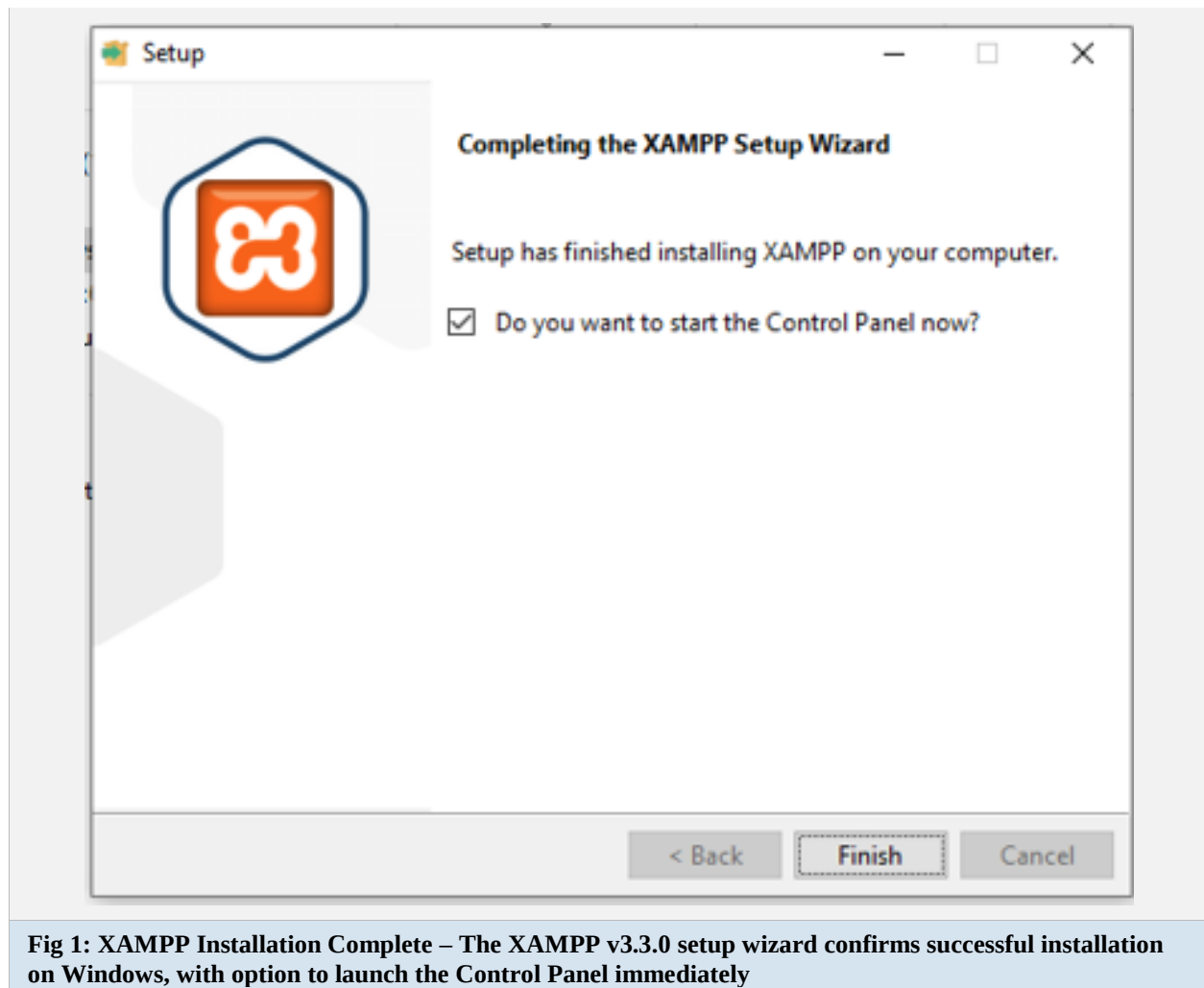


Fig 1: XAMPP Installation Complete – The XAMPP v3.3.0 setup wizard confirms successful installation on Windows, with option to launch the Control Panel immediately

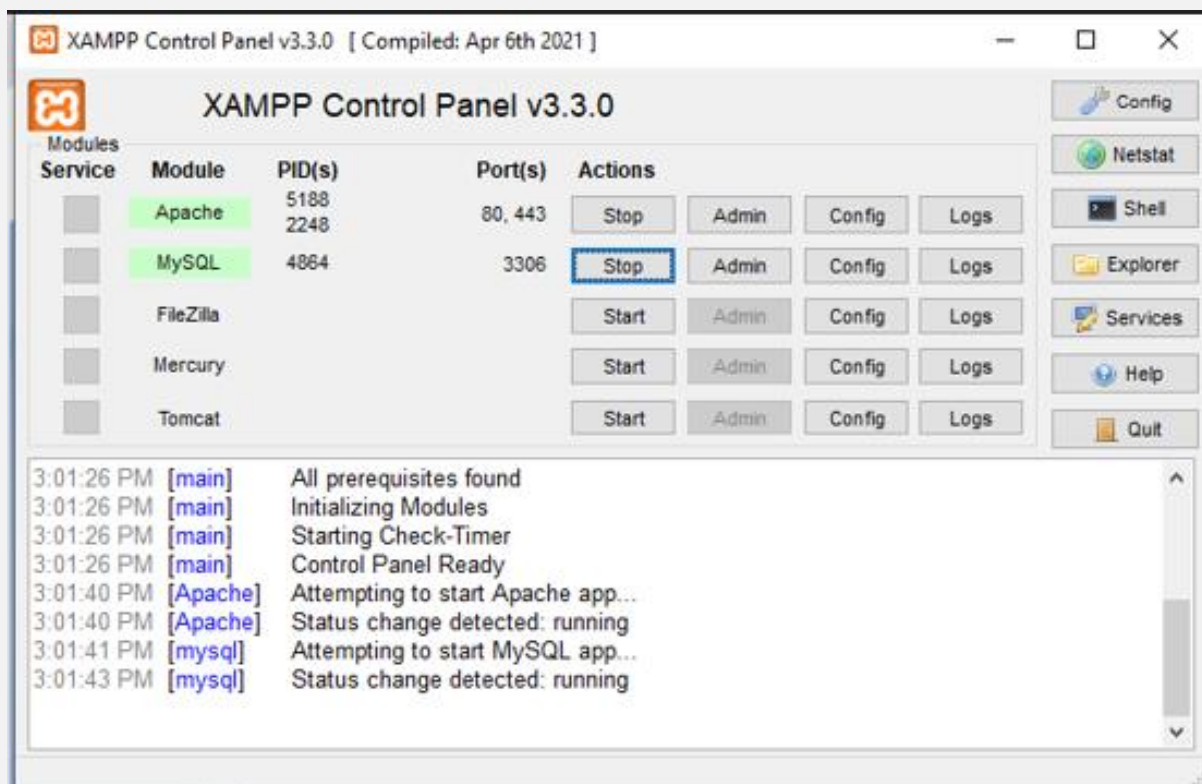


Fig 2: Apache and MySQL Running – XAMPP Control Panel v3.3.0 showing Apache running on ports 80/443 and MySQL on port 3306, with status logs confirming both services started successfully at 3:01 PM

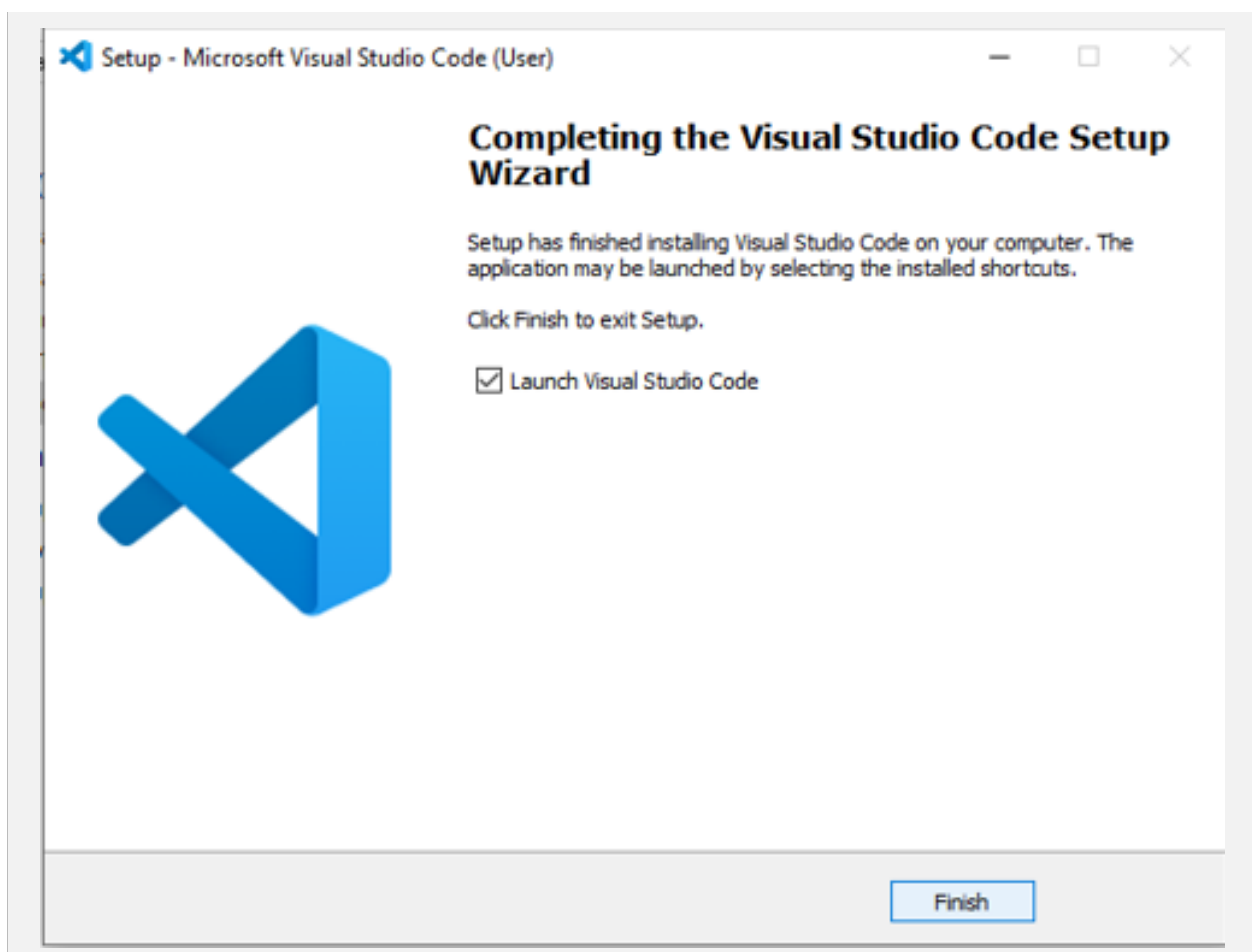


Fig 3: Laragon Alternative Server – Laragon 8.6.1 setup wizard completing installation of an alternative local development environment supporting Apache/Nginx, MySQL, Node.js, and Python alongside XAMPP

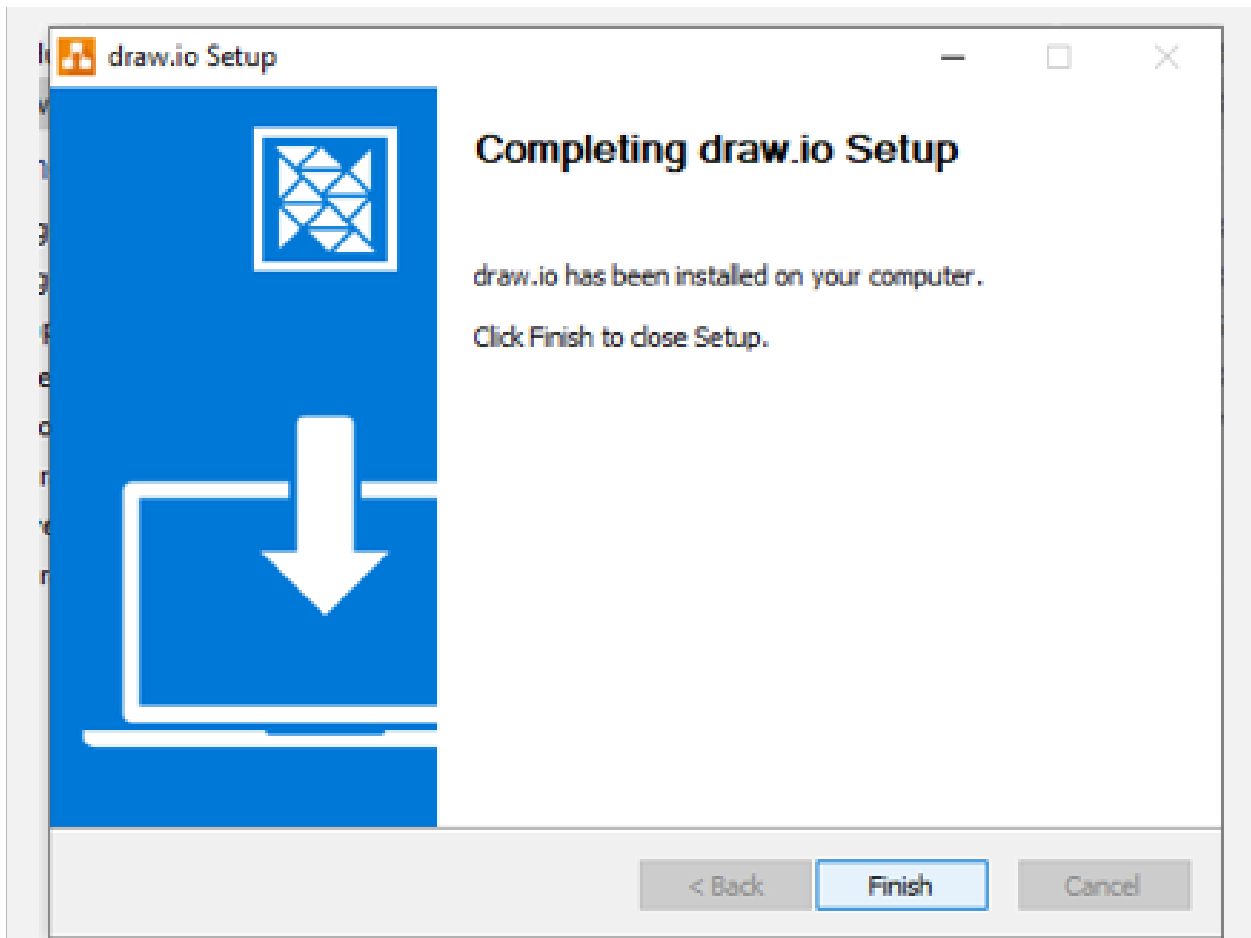


Fig 4: VS Code IDE Installed – Visual Studio Code setup wizard completing successfully; VS Code is the primary code editor used for writing all ShopMart PHP, HTML, and CSS files

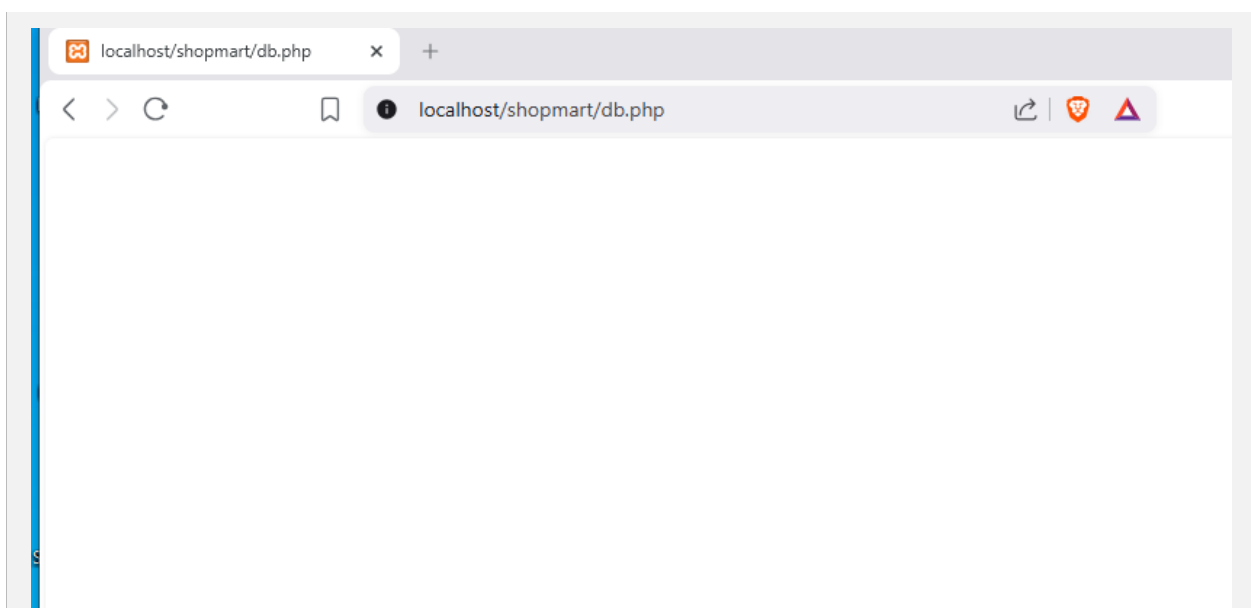


Fig 5: Database Connection Test – localhost/shopmart/db.php loads as a blank white page with no errors, confirming that PHP successfully connected to the MySQL shopmart database using mysqli_connect()

Student Reflection (Maximum 100 Words)

This week I set up my local development environment by installing XAMPP v3.3.0 — Fig 2 confirms Apache and MySQL were both running. VS Code was installed as the primary code editor and Laragon as a backup server. Fig 5 shows db.php loading blank with no errors, proving PHP connected to MySQL successfully. A port conflict on 443 was resolved by editing the XAMPP SSL config file. Technologies used: Apache, MySQL, PHP, phpMyAdmin, VS Code.

Week 2: Wireframes and GUI Design

Title

User Interface Planning and System Design

Required Evidence

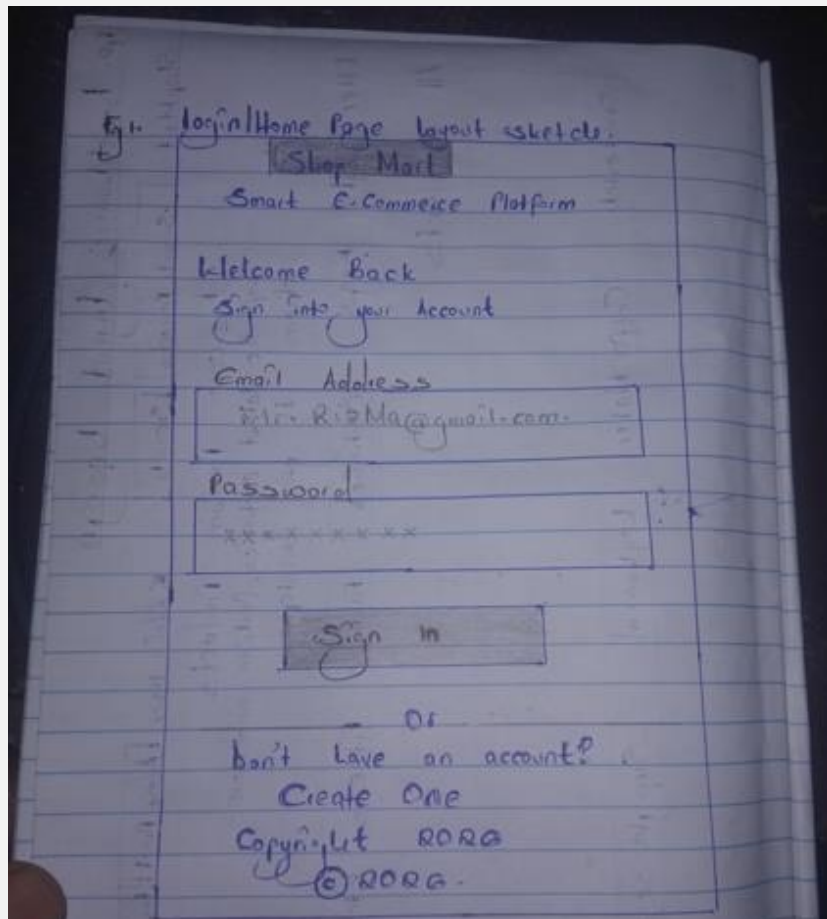


Fig 1: Hand-drawn Login/Homepage Wireframe – Sketch of the ShopMart login page showing the logo, 'Welcome Back' heading, Email Address field (Riz.Mac@gmail.com), Password field, Sign In button, and 'Don't have an account? Create One' link

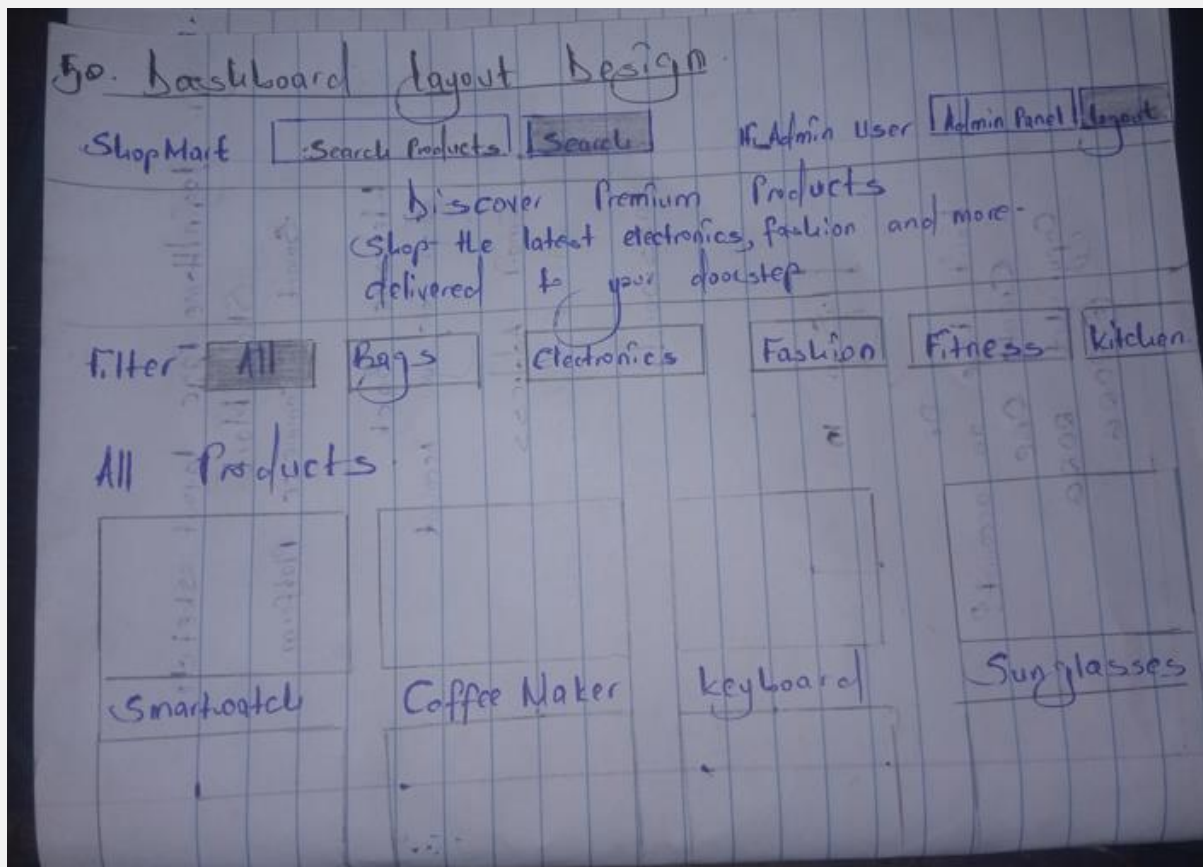


Fig 2: Dashboard Layout Design – Hand-drawn wireframe of the ShopMart homepage showing the full navbar (ShopMart logo, Search Products, Search button, Hi Admin User, Admin Panel, Logout), hero section, category filter bar (All, Bags, Electronics, Fashion, Fitness, Kitchen), and product grid with SmartWatch, Coffee Maker, Keyboard, Sunglasses

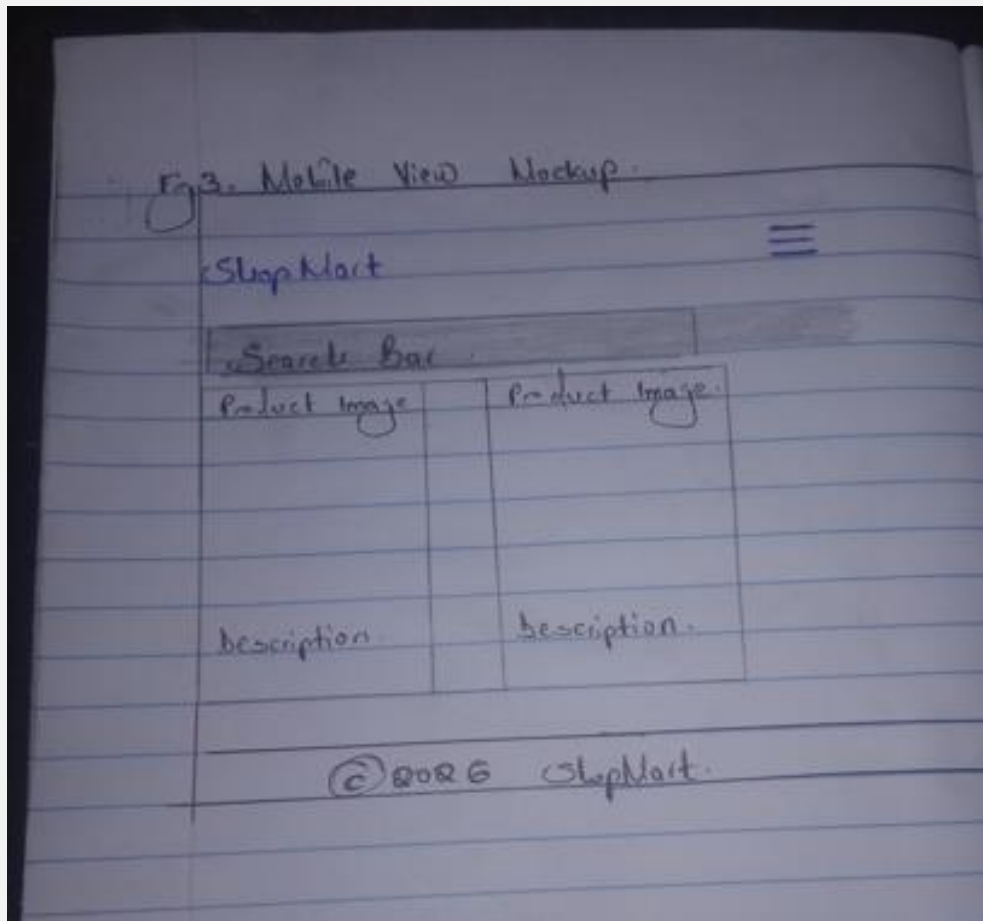


Fig 3: Mobile View Mockup – Hand-drawn mobile layout wireframe showing ShopMart's responsive design with hamburger menu, search bar, two-column product image grid with descriptions, and copyright footer

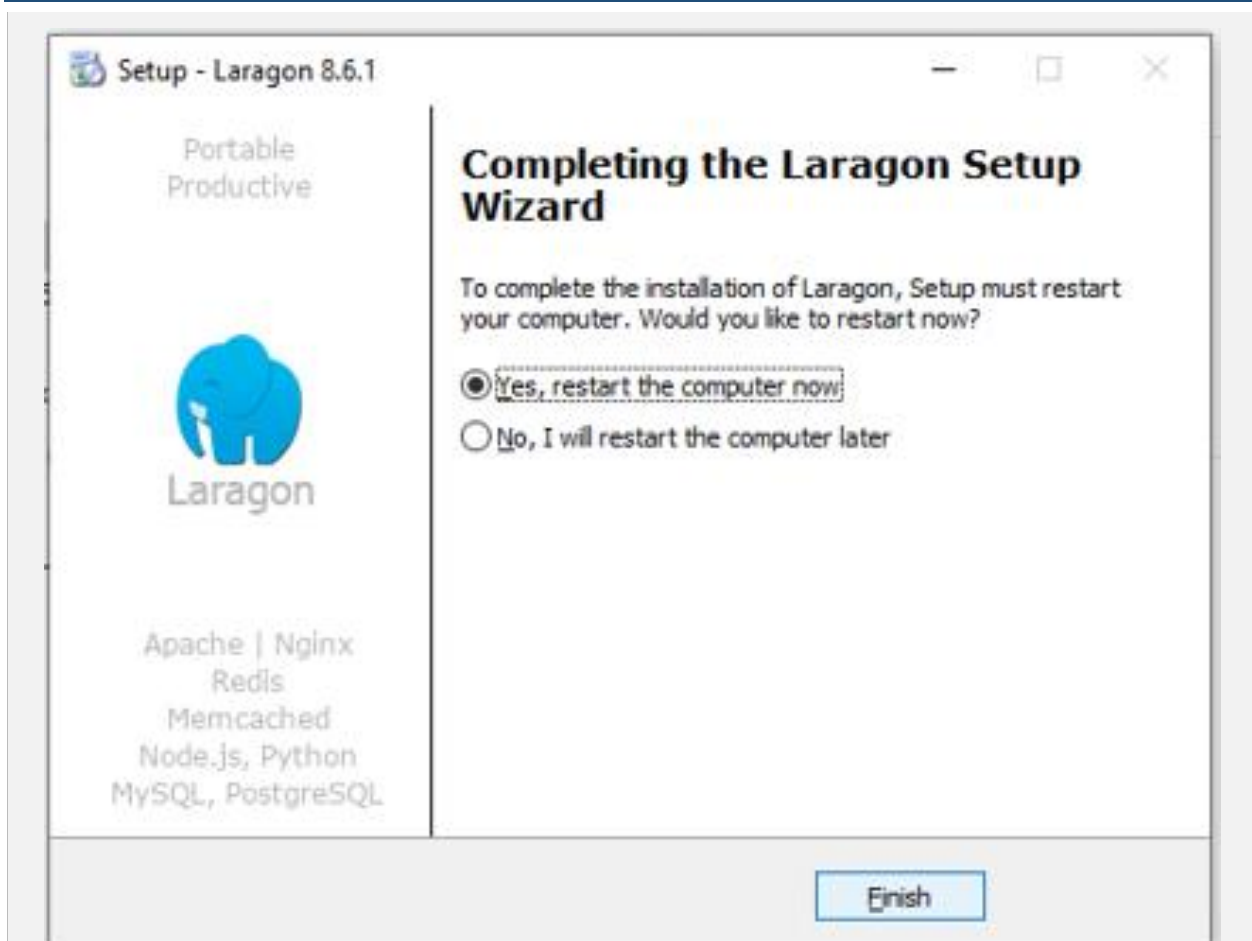


Fig 4: Color Theme and Navigation – draw.io desktop application successfully installed, the primary tool used for creating ShopMart digital wireframes, system flow diagrams, and GUI prototypes

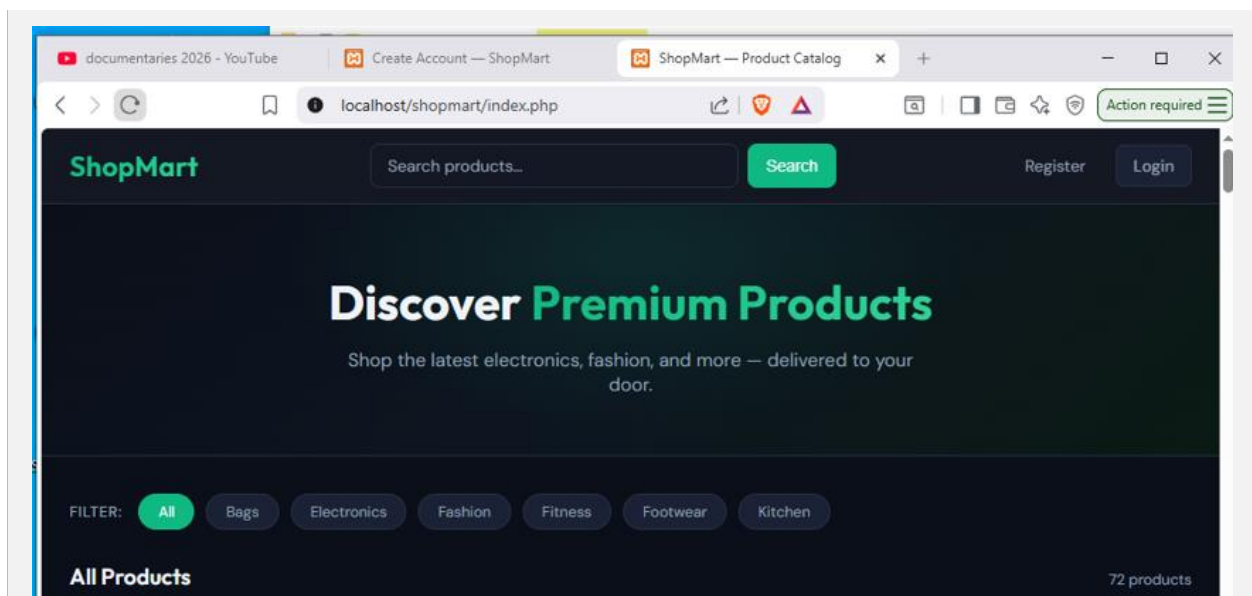


Fig 5: GUI Prototype – ShopMart storefront live at localhost/shopmart/index.php showing the 'Discover Premium Products' hero section, category filter bar with All/Bags/Electronics/Fashion/Fitness/Footwear/Kitchen buttons, and 72 products listed

Student Reflection (Maximum 100 Words)

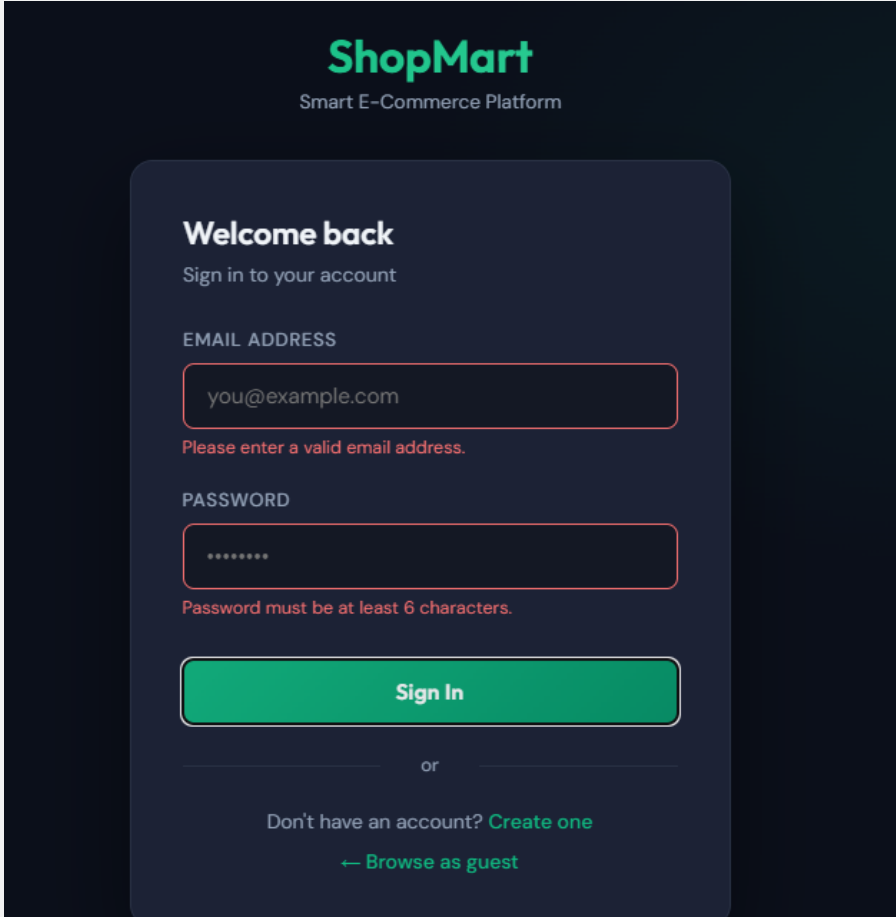
This week I planned the ShopMart interface before writing any code. I hand-drew the login wireframe and dashboard layout first, then used draw.io for digital prototypes. I chose a dark theme with teal (#10b981) accents for a modern professional look. The top navigation handles both admin and customer access. Category filters let users browse by product type. The design targets two user types: online shoppers and the admin managing inventory.

Week 3: JavaScript and PHP Basics

Title

Frontend Interaction and Backend Foundations

Required Evidence



The screenshot displays the ShopMart login interface. At the top, the ShopMart logo is shown in green, with the tagline 'Smart E-Commerce Platform' below it. The main heading is 'Welcome back', followed by the instruction 'Sign in to your account'. The form contains two input fields: 'EMAIL ADDRESS' and 'PASSWORD'. The email field contains 'you@example.com' and has a red error message below it: 'Please enter a valid email address.' The password field contains six dots and has a red error message below it: 'Password must be at least 6 characters.' A green 'Sign In' button is positioned below the password field. Below the button, there is a horizontal line with the word 'or' in the center. At the bottom, there is a link 'Don't have an account? Create one' and a link '← Browse as guest'.

Fig 1: JavaScript Form Validation – ShopMart login form showing two client-side validation error messages: 'Please enter a valid email address' and 'Password must be at least 6 characters', both triggered by JavaScript before any data is sent to PHP

Create account
Join ShopMart today — it's free

FULL NAME
e.g. Alice Kamau

EMAIL ADDRESS
you@example.com

PASSWORD
.....|
Fair

CONFIRM PASSWORD
Repeat your password

Create Account

Already have an account? [Sign in](#)

Fig 2: Password Strength Checker – ShopMart registration form showing a real-time password strength indicator displaying 'Fair' in orange as the user types, implemented using a JavaScript event listener on the password input field

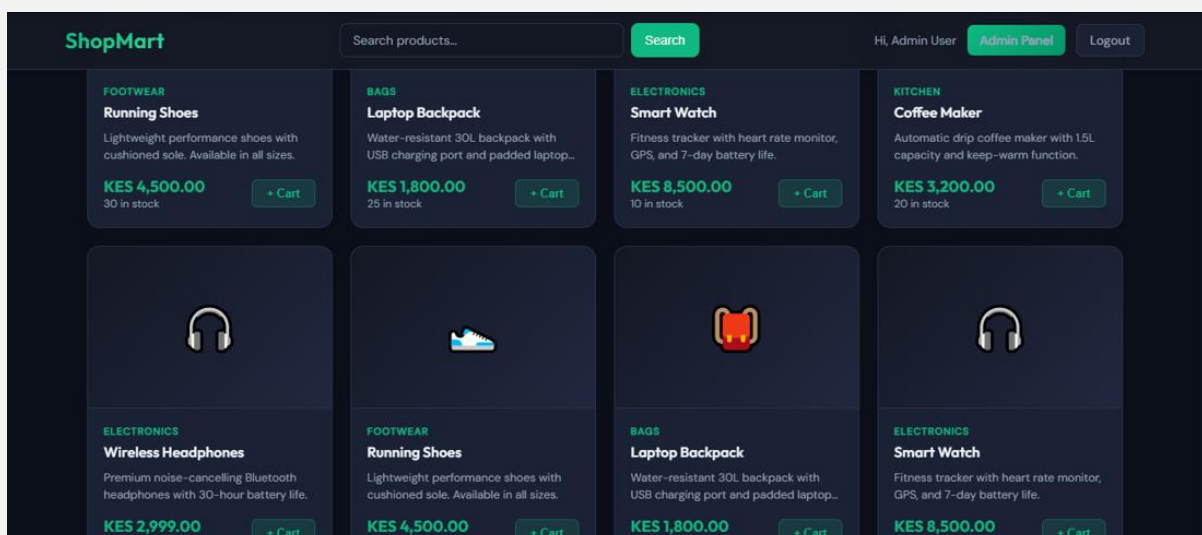


Fig 3: PHP Dynamic Product Output – ShopMart product catalog at localhost/shopmart/index.php dynamically rendering products (Running Shoes KES 4,500, Laptop Backpack KES 1,800, Smart Watch KES 8,500, Coffee Maker KES 3,200) fetched from MySQL, with 'Coffee Maker added to cart' toast

```

34     if (password_verify($password, $user['password'])) {
35         $_SESSION['user_id'] = $user['id'];
36         $_SESSION['name']    = $user['name'];
37         $_SESSION['email']   = $user['email'];
38         $_SESSION['role']    = $user['role'];
39
40         header('Location: ' . ($user['role'] === 'admin' ? 'admin.php' : 'index.php'));
41         exit;
42     }
43 }

```

Fig 4: PHP Session and Role-Based Redirect – VS Code showing the PHP authentication code: password_verify() check, \$_SESSION variable assignment (user_id, name, email, role), and conditional header redirect — admin goes to admin.php, customer goes to index.php

Fig 5: Dynamic User Input Handling – ShopMart Create Account page at localhost/shopmart/register.php with Full Name, Email Address, Password (min 6 chars), and Confirm Password fields processed server-side by PHP using \$_POST

Student Reflection (Maximum 100 Words)

This week I implemented JavaScript validation on the login form — Fig 1 shows two error messages appearing before the form reaches PHP. Fig 2 shows a live password strength indicator on the register page using JavaScript event listeners. PHP fetches products from MySQL dynamically as shown in Fig 3. Fig 4 shows the role-based redirect logic in VS Code using password_verify() and \$_SESSION. The main challenge was making both JS client-side and PHP server-side layers work together correctly.

Week 4: User Authentication

Title

Login, Registration, and Session Management

Required Evidence

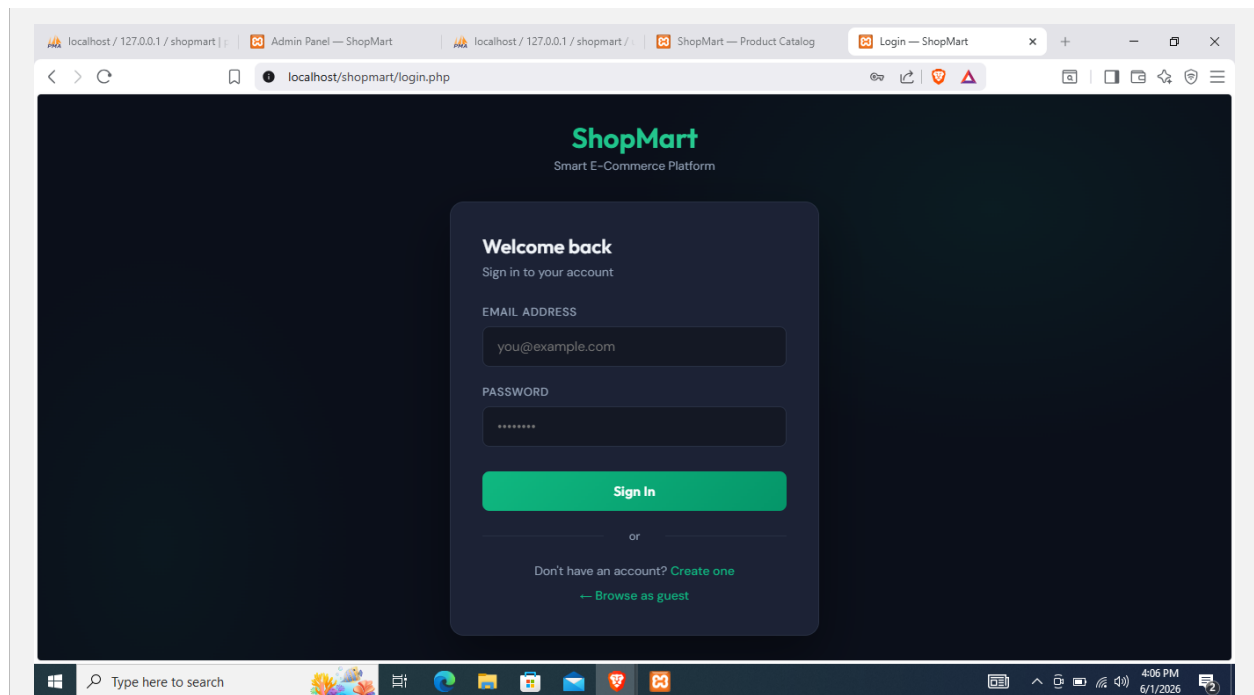
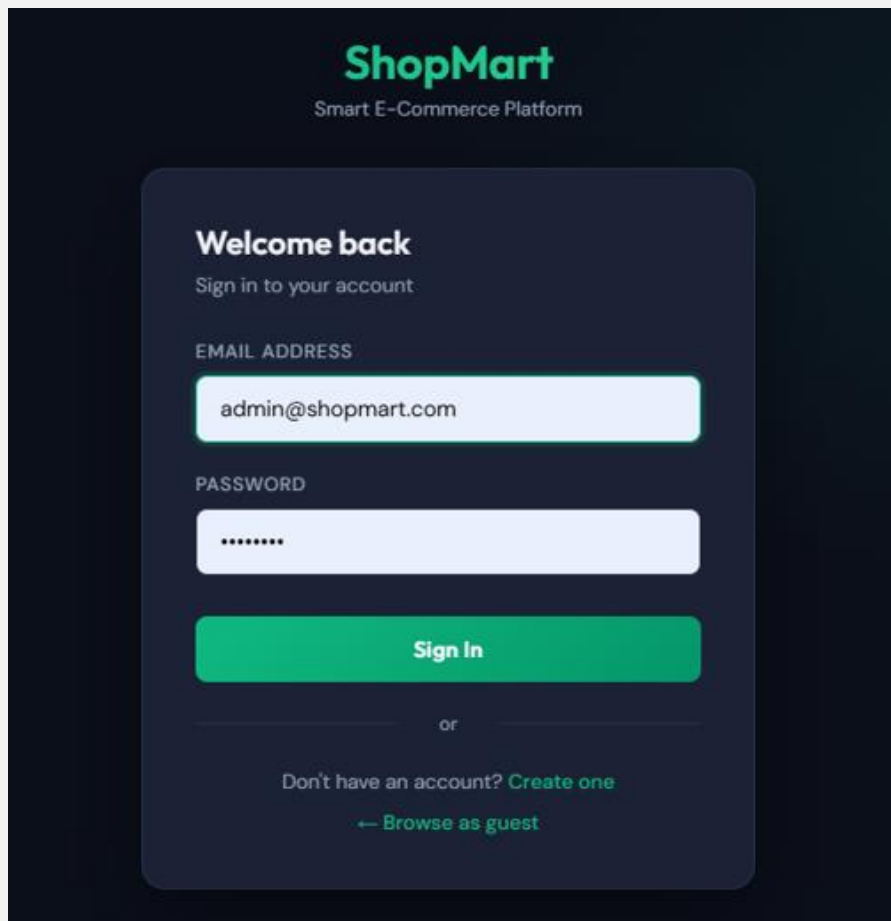


Fig 1: Login Page – ShopMart login form at localhost/shopmart/login.php showing the Email Address and Password fields, Sign In button, 'Don't have an account? Create one' link, and guest browsing option — the entry point for all user authentication



ShopMart
Smart E-Commerce Platform

Welcome back
Sign in to your account

EMAIL ADDRESS
admin@shopmart.com

PASSWORD
.....

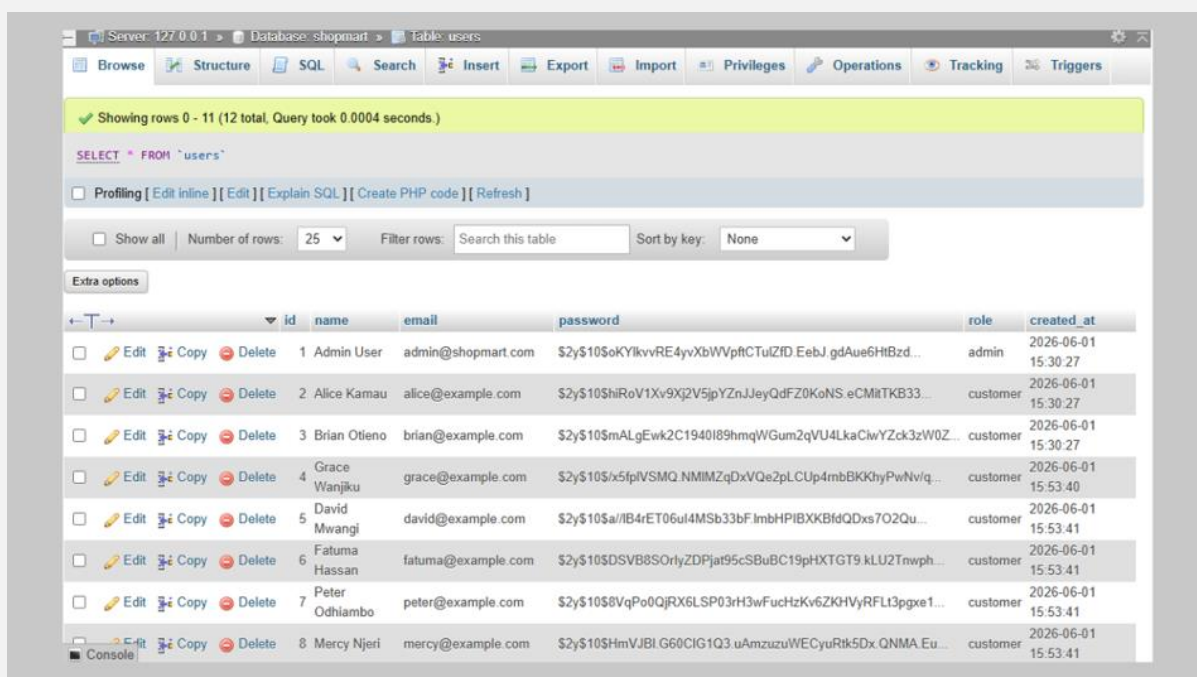
Sign In

or

Don't have an account? [Create one](#)

[← Browse as guest](#)

Fig 2: Login Form with Admin Credentials – Login page showing admin@shopmart.com entered in the Email Address field with password filled, demonstrating the admin login process that triggers role-based session creation



Server: 127.0.0.1 Database: shopmart Table: users

Showing rows 0 - 11 (12 total, Query took 0.0004 seconds.)

SELECT * FROM `users`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all Number of rows: 25 Filter rows: Search this table Sort by key: None

	id	name	email	password	role	created_at
☐	1	Admin User	admin@shopmart.com	\$2y\$10\$0KYLkvRE4yvXbWVpftCTulZID EebJ gdAue6HtBzd...	admin	2026-06-01 15:30:27
☐	2	Alice Kamau	alice@example.com	\$2y\$10\$hiRoV1xv9Xq2V5jpYZnJJeyQdFZ0KoNS eCMitKB33...	customer	2026-06-01 15:30:27
☐	3	Brian Otieno	brian@example.com	\$2y\$10\$mALgEwk2C1940i89hmqWGum2qVU4LkaCiwYZck3zW0Z...	customer	2026-06-01 15:30:27
☐	4	Grace Wanjiku	grace@example.com	\$2y\$10\$/x5fpIVSMQ NMIMZqDxVQe2pLCUp4mbBKKhyPwNv/q...	customer	2026-06-01 15:53:41
☐	5	David Mwangi	david@example.com	\$2y\$10\$a/IB4rET06ul4MSb33bFImbHPiBXKBfdQDxs7O2Qu...	customer	2026-06-01 15:53:41
☐	6	Fatuma Hassan	fatuma@example.com	\$2y\$10\$SDSVB8SOrlyZDPJat95cSBuBC19pHXTGT9 kLU2Tnwp...	customer	2026-06-01 15:53:41
☐	7	Peter Odhiambo	peter@example.com	\$2y\$10\$8VqPo0QjRX6LSP03rH3wFucHzKv6ZKHVYRfL13pgxe1...	customer	2026-06-01 15:53:41
☐	8	Mercy Njeri	mercy@example.com	\$2y\$10\$HmVJBI.G60CIG1Q3 uAmzuzuWECyurRtk5Dx QNMA Eu...	customer	2026-06-01 15:53:41

Fig 3: Bcrypt Password Hashing – phpMyAdmin users table at 127.0.0.1 showing 12 registered users (Admin User, Alice Kamau, Brian Otieno, etc.) with passwords stored as bcrypt hashes (\$2y\$10\$...) generated by PHP password_hash() — no plain text passwords stored

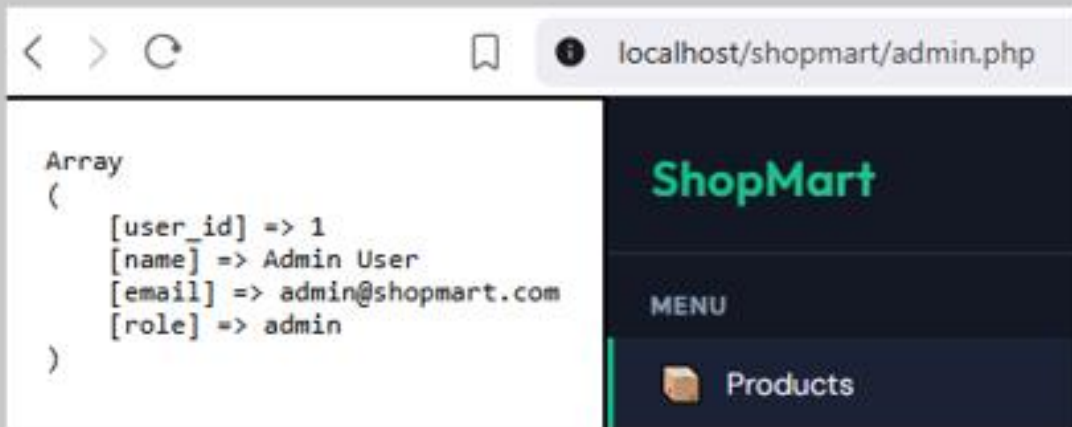


Fig 4: PHP Session Variables – Browser output showing the session array after successful login: [user_id]=>1, [name]=>Admin User, [email]=>admin@shopmart.com, [role]=>admin — confirming PHP session management works correctly

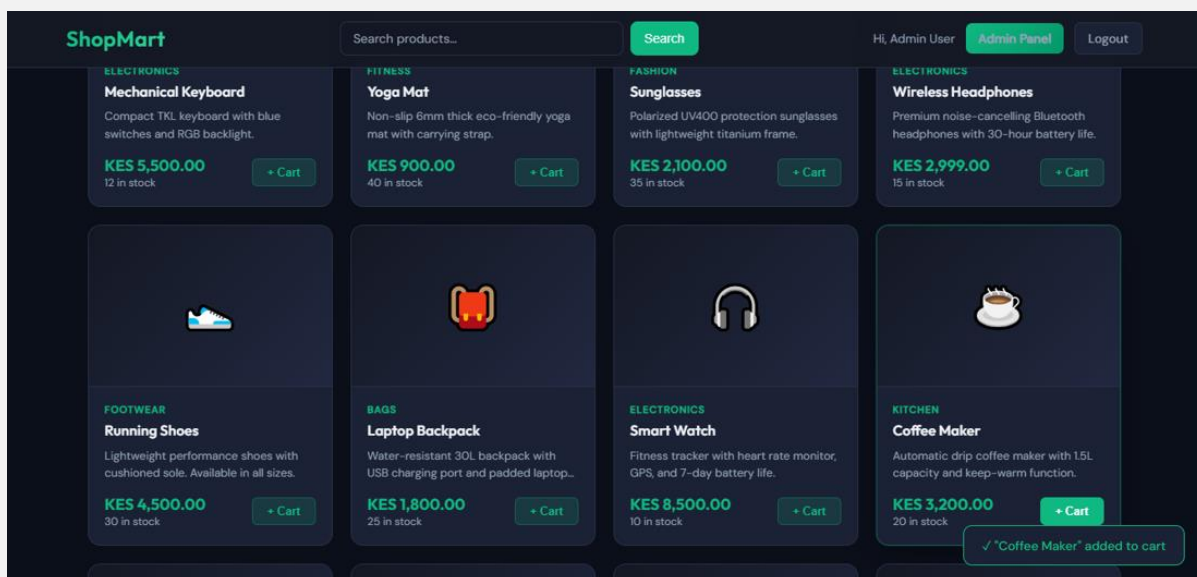


Fig 5: Role-Based Redirect to Admin Dashboard – localhost/shopmart/admin.php showing the Product Management dashboard with stats (72 Total Products, 12 Registered Users, 0 Low Stock) and Add Product form, accessible only to users with role=admin

Student Reflection (Maximum 100 Words)

This week I built ShopMart's full authentication system. Fig 3 shows 12 users in phpMyAdmin with passwords stored as bcrypt hashes using PHP password_hash() — no plain text is ever stored. Fig 4 shows the session array (user_id, name, email, role) set after successful login using password_verify(). Fig 5 confirms the role-based redirect sends admins to admin.php while customers go to index.php. CSRF tokens protect the login and registration forms against cross-site attacks.

Week 5: Product and Cart System

Title

E-Commerce Core Functionality

Required Evidence

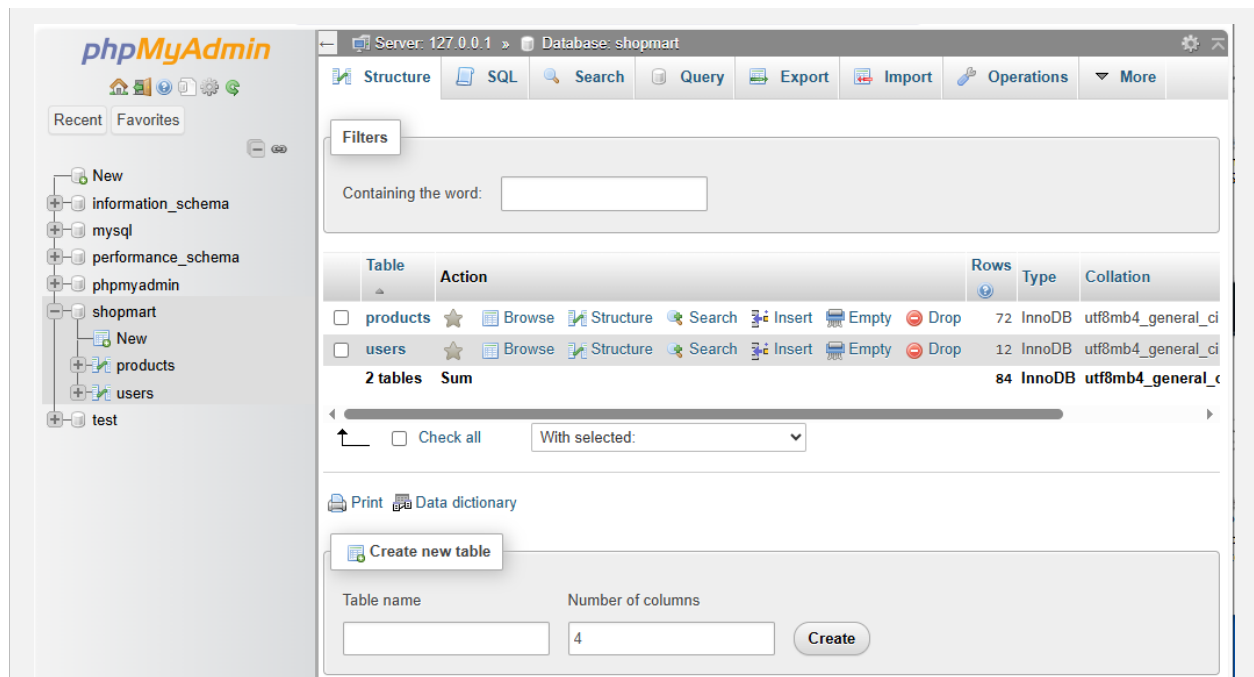


Fig 1: Product Listing Page – ShopMart product grid at localhost/shopmart/index.php dynamically rendering products fetched from MySQL: Running Shoes (KES 4,500), Laptop Backpack (KES 1,800), Smart Watch (KES 8,500), Coffee Maker (KES 3,200), Wireless Headphones (KES 2,999) with + Cart buttons and stock counts

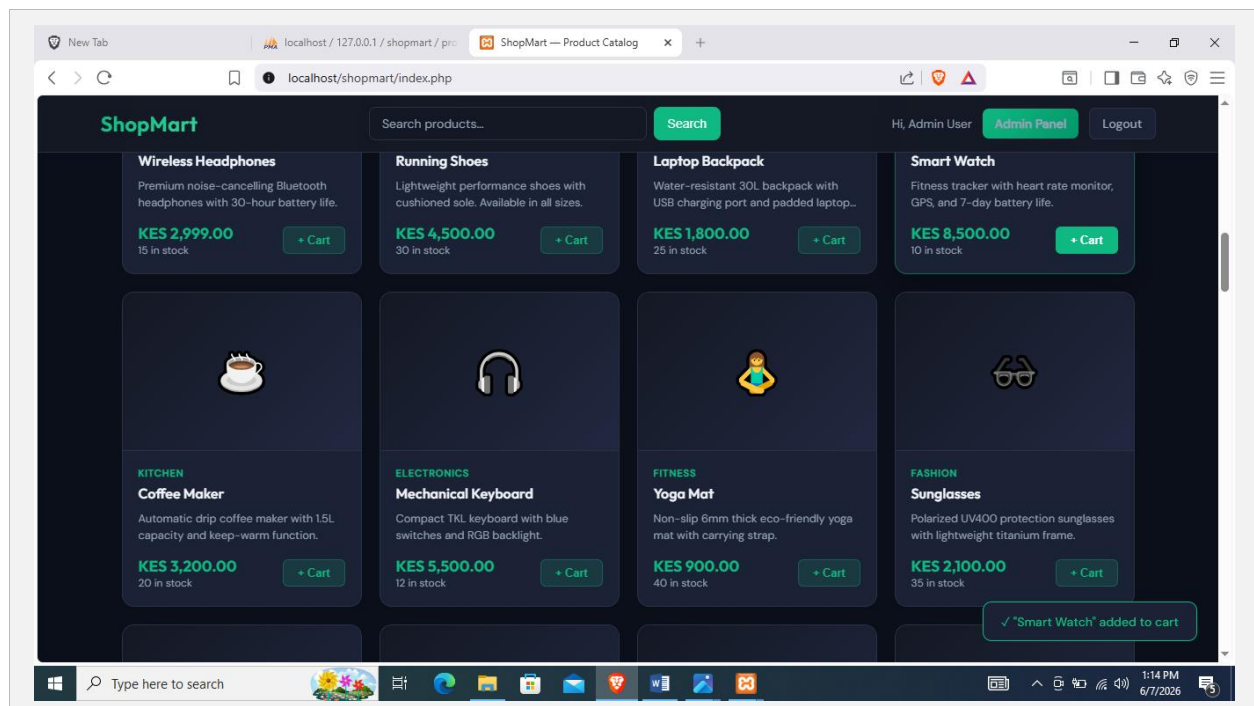


Fig 2: Add to Cart – Green toast notification 'Smart Watch added to cart' confirming that clicking + Cart POSTs the product_id to PHP, which saves the item in \$_SESSION['cart'] with quantity=1 — the session-based cart is working

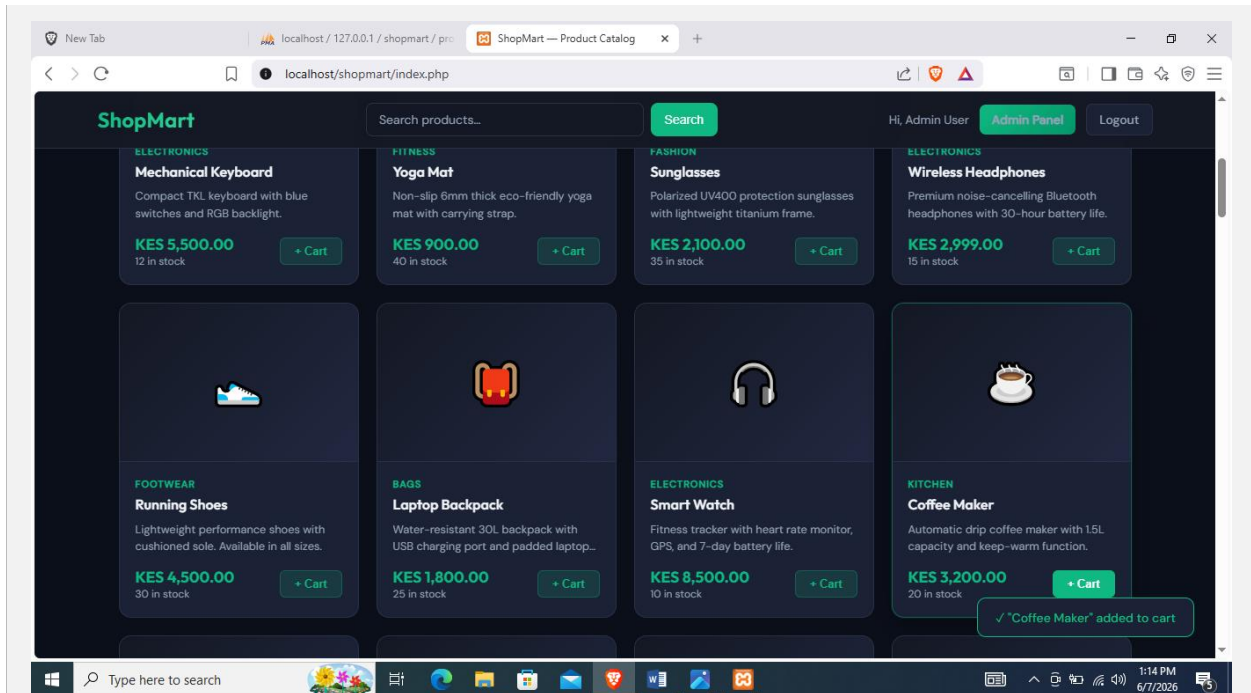


Fig 3: Cart Session Persisting – Product catalog showing 'Coffee Maker added to cart' toast while the admin is still logged in, confirming that \$_SESSION['cart'] data persists across different pages and does not reset during navigation

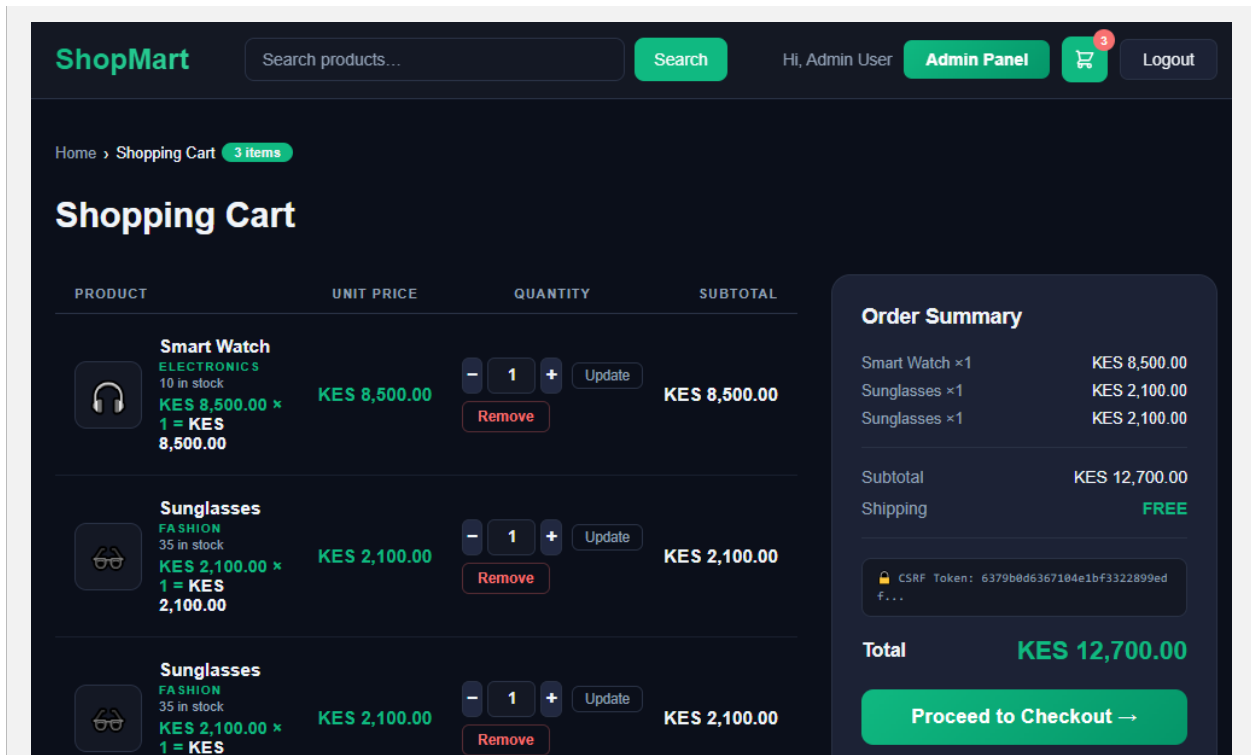


Fig 4: Cart Page – localhost/shopmart/cart.php showing 3 items (Smart Watch KES 8,500, Sunglasses KES 2,100, Sunglasses KES 2,100) with quantity steppers (–/+/Update), Remove buttons, Order Summary sidebar showing subtotal KES 12,700, FREE shipping, and Proceed to Checkout button

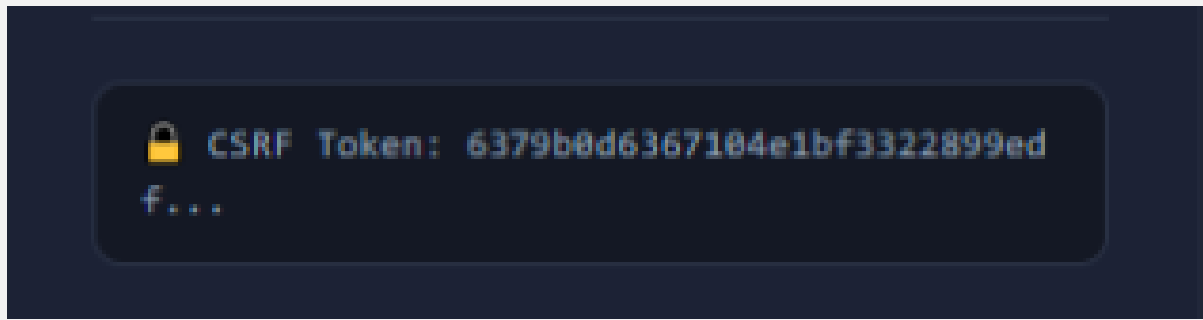


Fig 5: CSRF Protection – Order Summary panel in cart.php displaying the active CSRF token (6379b0d6367104e1bf3322899edf...) confirming that every cart form submission — update quantity, remove item, checkout — is validated against this server-generated token to prevent cross-site request forgery

Student Reflection (Maximum 100 Words)

This week I built the product listing and shopping cart for ShopMart. Fig 1 shows products rendered dynamically from MySQL. The + Cart button POSTs to PHP and saves items in \$_SESSION['cart'] as confirmed by the toast in Fig 2. Fig 3 shows the cart session persisting across pages without resetting. Fig 4 shows the complete cart page at localhost/shopmart/cart.php with quantity steppers, subtotals, and a total of KES 12,700. Fig 5 shows the CSRF token protecting all cart and checkout forms.